

MERP — How the repositories should be laid out

Proposal for team review · 14 August 2026

Status: proposal, not yet ratified. Nothing here should be created until the topology decision is signed off (see the last section).

1. The short version

Five active repositories. One frontend. One new repo to create.

- 1 Five repos, not eight: pil, merp, Axo, QUILL, vigil.
- 2 All user interface lives in one repo — merp. Every other repo is backend only.
- 3 The shared contracts live inside pil as a separately versioned package. Not a sixth repo. Never copy-pasted.
- 4 No repository without code in it. Otto, Guardrails and Pavo get folders when they have commits.

2. The five repositories

REPO	CONTAINS	WHAT IT HOLDS
pil	Backend (Go)	The floor. Shared contracts, the graph (ITKG), the ledger, the policy gate, the bus, the API gateway, all adapters, the merp verify CLI, and the Hermes edge runner.
merp NEW	Frontend (TypeScript)	The shell everyone logs into. Login, org, connections, capability grants, the product launcher, all product screens, the design system, and the deployment charts for the whole platform.
Axo	Backend	Remediation and fleet. API, orchestrator, verifier, executors, healing graph, osquery agent, outbox. Its current UI moves out to merp.
QUILL	Backend	Adjudication and control coverage. OSCAL export lives here and nowhere else.
vigil	Backend	Correlation. Produces state and evidence. Contains no remediation code, ever.

Inside pil — the floor

pil/	
├ contracts/	shared message shapes, closed vocabularies, the capability catalogue, code generation
	← versioned and tagged on its own
├ itkg/	the graph: schema, migrations, the only query path
├ ledger/	signed, chained evidence
├ gate/ policy/	the policy gate + the rules it runs
├ bus/	queue, orchestrator, rollback
├ gateway/	the one door: login, permissions, tenant scoping
├ adapters/	ScienceLogic · Prometheus · K8s · Fleet · AX0 sync
├ edge/hermes/	the edge runner
├ cli/merp/	merp verify · export · seed
└ docs/adr/	the spine decisions move here

Inside merp — the shell

merp/	
├ apps/console/	
├ shell/	top bar, launcher, role context, ledger tape
├ org/	connections, grants, members ← the demo lands here
├ lenses/	axo · vigil · quill · remi (folders, not repos)
├ design/	one set of tokens and components
├ api/	client generated from pil/contracts
├ deploy/	homelab k3s · k3d dev · demo tier
└ docs/	UX specs, role matrix, screenshots

Why the gateway sits in pil and not in merp : the gateway is where the tenant is read from the signed token and where permissions are enforced. That is backend security work. The shell is simply one of its clients — it never talks to a product directly.

3. Where the frontend goes

Today AXO has both a frontend and a backend in one repo. There are two clean ways forward. **We should take the second.**

	OPTION A — EACH PRODUCT KEEPS ITS OWN UI	OPTION B — ONE SHARED UI (RECOMMENDED)
Where UI lives	In each product repo; the shell loads them at runtime	One console in <code>merp</code> ; products expose APIs only
Setup needed	Module federation, pinned shared versions, a design system released across repos	A normal web app
Main risk	Version mismatch — shell v4 loading a lens built for gateway v2	Product work needs a change in <code>merp</code> to ship UI
Suits a team of	20+ engineers with a frontend owner per product	2-3 engineers — us

Option A buys independence we do not currently need, and charges integration work we cannot afford at this size. We can revisit it once a product has its own dedicated frontend engineer.

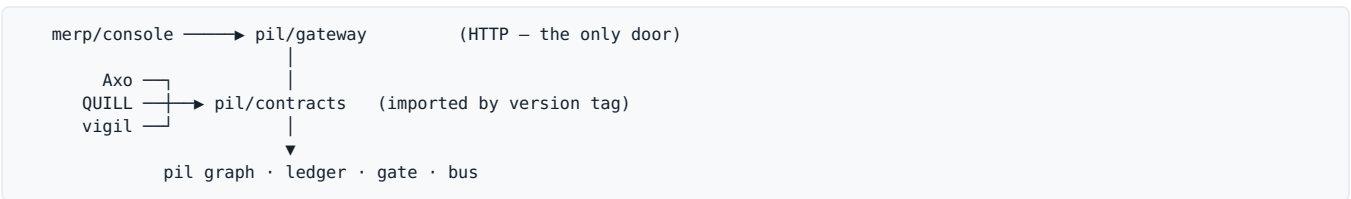
In practice: AXO's `desktop/src/pages/*` becomes `merp/apps/console/src/lenses/axo/`, and AXO becomes API-only. QUILL and vigil never grow a frontend at all.

4. What we are not creating

NAME	WHAT HAPPENS TO IT
Remi	Archive the repo. The disaster-recovery README in it is dead content. Remi is now a read-only posture score — some screens in <code>merp</code> plus one query in <code>pil/itkg</code> .
Otto	A roadmap name with no code. It becomes a folder in <code>Axo</code> when it exists — same execution path.
Guardrails	A roadmap name. It becomes a folder in <code>pil</code> when it exists.
Pavo	Roadmap. It owns the zero-trust posture surface — not <code>merp</code> 's, not Remi's.
Hermes	Not a product. It is the edge runner: <code>pil/edge/hermes</code> .
handbook	Leave it alone. Not part of the platform.
Gitea (192.168.4.80)	A staging mirror. Never the source of truth. Nothing starts life there.

Empty placeholder repos are how a project ends up with seven READMEs each describing a different architecture.

5. Four rules the build should enforce



- **No product imports another product.** Not AXO to QUILL, not QUILL to vigil, not any pair. They cooperate by leaving findings, change records and evidence in the shared stores. That is what lets us add an eighth product without touching the other seven.
- **Nothing reads the graph directly.** All graph access goes through the tenant-scoped query library in `pil/itkg`.
- **The console never calls a product API directly.** Everything goes through the gateway.
- **Contracts are consumed by version tag,** never vendored or copied into a product repo.

Add a `make deps-check` step to CI so these fail the build rather than depending on someone catching them in review.

6. Order of work

- **1 Merge PR #46 and #47.** The contracts package and the ITKG work are unlanded until then. Nothing else is safe to build on top of an unmerged spine.
- **2 Merge-day triage.** 21 commits on AXO `main` to review, roughly 8 files expected to conflict — including `circuit_breaker.py`, where `check_circuit` now returns a tuple on the spine branch.
- **3 IDI-23** (signature verification), then **IDI-19** (tenant isolation). Both P0.
- **4 Sign off the topology** — see below.
- **5 Then** create `merp`, move the console into it, and strip AXO's frontend.

7. Two things still open — please read before acting

This document contradicts the agent handover. Section 4.1 of the handover specifies a *single* monorepo called `merp-platform`, with AXO inside it as a service. What is described here is the multi-repo shape we are actually building. Both are defensible; they cannot both be true. Once PRs #46/#47 land, read what those ADRs say about repository layout. If they do not settle it, this becomes the topology ADR and needs sign-off **before** anyone creates the `merp` repo.

ADR-17 is unresolved — sovereign/air-gapped versus AWS — and it blocks PIL module work. The console is one of the few things that can proceed in parallel, but only if it assumes nothing about where it is deployed: a relative URL to the gateway, no cloud-specific login, no dependency that only exists in a hosted environment.

Prepared for team discussion. Comments and disagreement welcome before this is turned into an ADR.